MANIPAL SCHOOL OF INFORMATION SCIENCES
CRYPTOLOGY LAB — ASSIGNMENT 8

RSA Algorithm and Diffie-Hellman Key Exchange

Student Name: Ribhav Devprasad
Registration No: 261100690002
Course: Cryptology Lab

Q1. RSA Algorithm — Key Generation, Encryption and Decryption

RSA picks two prime numbers p and q, computes n = p×q and $\phi(n) = (p-1)(q-1)$, chooses a public exponent e coprime with $\phi(n)$, and derives the private exponent d as the modular inverse of e mod $\phi(n)$. A message m is encrypted as $c = m^e \bmod n$ and decrypted as $m = c^d \bmod n$. Small primes are used here so the demonstration is readable; real RSA uses primes hundreds of digits long.

```
import random
from math import gcd

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def generate_prime(low, high):
    candidates = [n for n in range(low, high) if is_prime(n)]
    return random.choice(candidates)

def generate_keypair():
    p = generate_prime(100, 300)
    q = generate_prime(100, 300)
    while q == p:
        q = generate_prime(100, 300)
    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537 if gcd(65537, phi) == 1 else 3
    while gcd(e, phi) != 1:
        e += 2
    d = pow(e, -1, phi)
    return (e, n), (d, n), p, q

def rsa_encrypt(message, public_key):
    e, n = public_key
    return [pow(ord(ch), e, n) for ch in message]

def rsa_decrypt(cipher_nums, private_key):
    d, n = private_key
    return "".join(chr(pow(c, d, n)) for c in cipher_nums)

public_key, private_key, p, q = generate_keypair()
print(f"Primes chosen : p = {p}, q = {q}")
```

```
print(f"Public key  (e, n) : {public_key}")
print(f"Private key (d, n) : {private_key}")

message = input("Enter message to encrypt: ")
cipher_nums = rsa_encrypt(message, public_key)
print("Ciphertext (as numbers):", cipher_nums)

decrypted = rsa_decrypt(cipher_nums, private_key)
print("Decrypted message:", decrypted)
```

Sample Run

```
Primes chosen : p = 137, q = 103
Public key  (e, n) : (65537, 14111)
Private key (d, n) : (13745, 14111)
Enter message to encrypt: HI
Ciphertext (as numbers): [5184, 11357]
Decrypted message: HI
```

Q2. Diffie-Hellman (DH) Key Exchange

Diffie-Hellman lets two parties agree on a shared secret over a public channel without ever sending the secret itself. Both sides agree on a public prime p and a primitive root (generator) g. Each side picks a private random number, computes a public value from it, and exchanges only that public value. Because discrete logarithms are hard to reverse, an eavesdropper who sees p, g and both public values still cannot efficiently recover the private numbers or the shared secret.

```
import random

def generate_private_key(p):
    return random.randint(2, p - 2)

def compute_public_key(g, private_key, p):
    return pow(g, private_key, p)

def compute_shared_secret(other_public_key, private_key, p):
    return pow(other_public_key, private_key, p)

# Publicly agreed parameters
p = 7919     # a prime number (public)
g = 3        # a primitive root of p (public)
print(f"Public prime (p): {p}")
print(f"Public base  (g): {g}")

# Alice's side
alice_private = generate_private_key(p)
alice_public = compute_public_key(g, alice_private, p)

# Bob's side
bob_private = generate_private_key(p)
bob_public = compute_public_key(g, bob_private, p)

print(f"Alice's public value sent to Bob : {alice_public}")
print(f"Bob's public value sent to Alice : {bob_public}")

# Each side computes the same shared secret independently
alice_shared_secret = compute_shared_secret(bob_public, alice_private, p)
bob_shared_secret = compute_shared_secret(alice_public, bob_private, p)
```

```
print("Alice's computed shared secret:", alice_shared_secret)
print("Bob's computed shared secret  :", bob_shared_secret)
print("Keys match:", alice_shared_secret == bob_shared_secret)
```

Sample Run

```
Public prime (p): 7919
Public base  (g): 3
Alice's public value sent to Bob : 6683
Bob's public value sent to Alice : 2946
Alice's computed shared secret: 5662
Bob's computed shared secret  : 5662
Keys match: True
```

Bonus: RSA Digital Signatures (Authentication, not just Secrecy)

RSA encryption with the public key keeps a message secret; using RSA the other way round — “encrypting” a message's hash with the sender's private key — proves who sent it. Anyone with the sender's public key can verify the signature, but only the sender's private key could have created it, so the message cannot be forged by someone who only has the public key.

```
import hashlib

def rsa_sign(message, private_key):
    d, n = private_key
    digest = int(hashlib.sha256(message.encode()).hexdigest(), 16) % n
    return pow(digest, d, n)

def rsa_verify(message, signature, public_key):
    e, n = public_key
    digest = int(hashlib.sha256(message.encode()).hexdigest(), 16) % n
    recovered = pow(signature, e, n)
    return recovered == digest

message = "Transfer approved by Ribhav"
signature = rsa_sign(message, private_key)
print("Signature:", signature)
print("Verifies with correct public key :", rsa_verify(message, signature, public_key))

tampered = "Transfer approved by Attacker"
print("Verifies tampered message        :", rsa_verify(tampered, signature, public_key))
```

Sample Run (using the key pair generated in Q1)

```
Signature: 12447
Verifies with correct public key : True
Verifies tampered message        : False
```

The second check fails because changing even one character of the message changes its SHA-256 hash completely, so the recovered digest no longer matches — this is what lets a verifier detect tampering as well as confirm the sender's identity.

Bonus: Why Plain Diffie-Hellman Still Needs Authentication

The Q2 exchange proves Alice and Bob agree on a secret, but on its own it does not prove who is on the other end. A man-in-the-middle (commonly “Eve”) can intercept both public values, run the protocol separately with each side, and end up holding two different shared secrets — one with Alice, one with Bob — without either noticing. In practice this is prevented by combining DH with

authentication, for example by signing the public values with RSA (as above) or using certificates, which is exactly how TLS combines DH key exchange with RSA/ECDSA signatures.

2.1 Why the plain exchange is secure against eavesdropping (though not impersonation)

- Only **p**, **g**, and the two public values ever travel over the channel; the private numbers and the shared secret itself are never transmitted.
- Computing the shared secret from the public values alone requires solving the discrete logarithm problem, which has no known efficient solution for large primes.
- Both parties arrive at the identical shared secret independently, which can then be used as a symmetric key (for example, for AES) for the rest of the conversation.